Palauta ratkaisusi Moodlessa viimeistään **sunnuntaina 19.4. kello 23:59**. Ratkaisujesi täytyy sisältää välivaiheita ja perusteluja.

Harjoitus 7.1. Olkoot g ja h derivoituvia funktioita. Kirjoita seuraavien funktioiden derivaatat käyttäen alkeisfunktioita ja funktioita $g(x)$, $h(x)$, $g'(x)$, $h'(x)$:

(a) $f(x) = g(x)^2 + h(x)$

(b) $f(x) = g(h(x))^2 + h(h(x))$

(c) $f(x) = g(x^3) + h(\cos(x))$

(d) $f(x) = h(g(x)^2) + \exp(h(x))$

Esimerkki: Ketjusäännön ja kaavan $\frac{\mathrm{d}}{\mathrm{d}x}\sin(x) = \cos(x)$ nojalla funktion $f(x) = 2\sin(g(x))$ derivaatta on $f'(x) = 2\cos(g(x))g'(x)$.

Ratkaisu. Palauta mieleen *Differentiaalilaskenta*-kurssin peruskaavat, kuten

$$\text{Ketjusääntö:} \quad D(f \circ g)(x) = (Df \circ g)(x) \cdot Dg(x) \quad \text{joillekin derivoituville funktioille } f, g,$$
$$\text{Derivoinnin lineaarisuus:} \quad D[af(x) + bg(x)] = aDf(x) + bDg(x) \quad \text{joillekin } a, b \in \mathbb{R},$$

sekä alkeisfunktioiden tunnetut derivaatat.

(a)

$$\begin{aligned}
Df(x) &= D[g(x)^2 + h(x)] \\
&= D[g(x)^2] + Dh(x) && \text{derivoinnin lineaarisuuden nojalla} \\
&= D(t^2 \circ g)(x) + Dh(x) \\
&= (D[t^2] \circ g)(x) \cdot Dg(x) + Dh(x) && \text{ketjusäännön nojalla} \\
&= (2t \circ g)(x) Dg(x) + Dh(x) && \text{käyttäen } D[t^n] = nt^{n-1} \\
&= 2g(x)Dg(x) + Dh(x)
\end{aligned}$$

(b)

$$\begin{aligned}
Df(x) &= D[g(h(x))^2 + h(h(x))] \\
&= D[g(h(x))^2] + D[h(h(x))] && \text{derivoinnin lineaarisuuden nojalla} \\
&= D(t^2 \circ g \circ h)(x) + D(h \circ h)(x) \\
&= (D[t^2] \circ (g \circ h))(x) \cdot D(g \circ h)(x) + (Dh \circ h)(x) \cdot Dh(x) && \text{ketjusäännön nojalla} \\
&= (D[t^2] \circ g \circ h)(x) \cdot (Dg \circ h)(x) \cdot Dh(x) + (Dh \circ h)(x) \cdot Dh(x) && \text{ketjusäännön nojalla} \\
&= (2t \circ g \circ h)(x)(Dg \circ h)(x) Dh(x) + (Dh \circ h)(x) Dh(x) && \text{käyttäen } D[t^n] = nt^{n-1} \\
&= 2g(h(x))Dg(h(x))Dh(x) + Dh(h(x))Dh(x)
\end{aligned}$$

(c)

$$\begin{aligned}
Df(x) &= D[g(x^3) + h(\cos(x))] \\
&= D[g(x^3)] + D[h(\cos(x))] && \text{derivoinnin lineaarisuuden nojalla} \\
&= D(g \circ t^3)(x) + D(h \circ \cos)(x) \\
&= (Dg \circ t^3)(x) \cdot D[x^3] + (Dh \circ \cos)(x) \cdot D[\cos(x)] && \text{ketjusäännön nojalla} \\
&= (Dg \circ t^3)(x) \cdot 3x^2 + (Dh \circ \cos)(x) \cdot D[\cos(x)] && \text{käyttäen } D[t^n] = nt^{n-1} \\
&= (Dg \circ t^3)(x) \cdot 3x^2 + (Dh \circ \cos)(x) \cdot (-\sin(x)) && \text{käyttäen } D[\cos(t)] = -\sin(t) \\
&= 3Dg(x^3)x^2 - Dh(\cos(x))\sin(x)
\end{aligned}$$

(d)

$$\begin{aligned}
Df(x) &= D[h(g(x)^2) + \exp(h(x))] \\
&= D[h(g(x)^2)] + D[\exp(h(x))] && \text{derivoinnin lineaarisuuden nojalla} \\
&= D(h \circ t^2 \circ g)(x) + D(\exp \circ h)(x) \\
&= (Dh \circ (t^2 \circ g))(x) D(t^2 \circ g)(x) + (D\exp \circ h)(x) Dh(x) && \text{ketjusäännön nojalla} \\
&= (Dh \circ t^2 \circ g)(x) D(t^2 \circ g)(x) + (\exp \circ h)(x) Dh(x) && \text{käyttäen } D\exp(t) = \exp(t) \\
&= (Dh \circ t^2 \circ g)(x)(2t \circ g)(x) Dg(x) + (\exp \circ h)(x) Dh(x) && D(t^2 \circ g)(x) = (2t \circ g)(x) Dg(x) \\
&= 2Dh(g(x)^2) g(x) Dg(x) + \exp(h(x)) Dh(x)
\end{aligned}$$

□

Harjoitus 7.2. Jokaisella seuraavista funktioista on kriittinen piste $x^* = 0$:

(a) $f(x) = 1 - x^2$

(b) $f(x) = x^2 - 1000x^{10}$

(c) $f(x) = x^3 e^{-x}$

(d) $f(x) = x^2 - (\cos(x))^2$

Onko $x^* = 0$ funktion lokaali ääriarvopiste vai ei? Jos se on ääriarvopiste, onko se funktion minimi- vai maksimipiste? Perustele vastauksesi tutkimalla, miten derivaatat käyttäytyvät pisteen $x = 0$ ympäristössä.

Ratkaisu.

(a) $f(x) = 1 - x^2$. Derivaatta on

$$f'(x) = -2x.$$

Koska derivaatta on positiivinen välillä $(-\infty, 0)$ ja negatiivinen välillä $(0, +\infty)$, funktio on aidosti kasvava pisteen $x^* = 0$ vasemmalla puolella ja aidosti vähenevä sen oikealla puolella. Näin ollen x^* on lokaali (tässä tapauksessa myös globaali) *maksimipiste*.

(b) $f(x) = x^2 - 1000x^{10}$. Derivaatta on

$$f'(x) = 2x - 10000x^9 = (2 - 10000x^8)x.$$

Tekijä x on positiivinen, kun $x > 0$. Tekijä $2 - 10000x^8$ on positiivinen jos ja vain jos

$$|x| < a := \left(\tfrac{2}{10000}\right)^{1/8}.$$

Näin ollen tulon nollasäännön nojalla

	$(-\infty,-a)$	$-a$	$(-a,0)$	0	$(0,a)$	a	$(a,+\infty)$
x	$-$	$-$	$-$	0	$+$	$+$	$+$
$2-10000x^8$	$-$	0	$+$	$+$	$+$	0	$-$
$f'(x)$	$+$	0	$-$	0	$+$	0	$-$
$f(x)$	$\nearrow$	M	$\searrow$	m	$\nearrow$	M	$\searrow$

derivaatta on negatiivinen, kun $x \in (-a,0)$, ja positiivinen, kun $x \in (0,a)$. Näin ollen funktio on aidosti vähenevä pisteen $x^* = 0$ vasemmalla puolella ja aidosti kasvava sen oikealla puolella. Näin ollen x^* on lokaali *minimipiste*. (Tällä funktiolla ei ole globaalia minimipistettä.)

(c) $f(x) = x^3 e^{-x}$. Derivaatta on

$$f'(x) = 3x^2 e^{-x} + x^3 \cdot (-e^{-x}) = (3-x)x^2 e^{-x}.$$

Tekijä $x^2 e^{-x}$ on aina epänegatiivinen, kun taas $3-x$ on positiivinen, jos $x < 3$, ja negatiivinen, jos $x > 3$. Näin ollen derivaatta on positiivinen sekä välillä $(-\infty,0)$ että välillä $(0,3)$, mikä tarkoittaa, että funktio on aidosti kasvava sekä pisteen $x^* = 0$ vasemmalla että oikealla puolella. Näin ollen kriittinen piste $x^* = 0$ *ei ole ääriarvopiste*.

(d) $f(x) = x^2 - (\cos(x))^2$. Derivaatta on

$$f'(x) = 2x + 2\cos(x)\sin(x) = 2(x + \cos(x)\sin(x)).$$

Palauta mieleen trigonometriasta, että $\cos(x) > 0$, jos $|x| < \pi/2$, ja $\sin(x) > 0$, jos $0 < x < \pi/2$, ja $\sin(x) < 0$, jos $-\pi/2 < x < 0$. Tämä tarkoittaa, että $\operatorname{sgn}\big[\cos(x)\sin(x)\big] = \operatorname{sgn}[\sin(x)]$, kun $|x| < \pi/2$. Lisäksi huomaa, että $\operatorname{sgn}[\sin(x)] = \operatorname{sgn}[x]$, kun $|x| < \pi/2$. Näin ollen $x + \cos(x)\sin(x) < 0$, jos $-\pi/2 < x < 0$, ja $x + \cos(x)\sin(x) > 0$, jos $0 < x < \pi/2$. Näin ollen funktio on aidosti vähenevä pisteen $x^* = 0$ vasemmalla puolella ja aidosti kasvava sen oikealla puolella. Näin ollen x^* on lokaali (tässä tapauksessa myös globaali) *minimipiste*.

Huomaa, että sgn on signum-funktio ja se määritellään $\operatorname{sgn}(x) := \begin{cases} -1 & \text{jos } x < 0, \\ 0 & \text{jos } x = 0, \\ 1 & \text{jos } x > 0. \end{cases}$

□

Harjoitus 7.3. Olkoon $C > 0$ ja oletetaan, että f on derivoituva funktio, joka toteuttaa epäyhtälön

$$f(x) - f(y) - f'(y)(x-y) \le C(x-y)^2 \quad \text{kaikilla} \quad x,y \in \mathbb{R}. \tag{1}$$

Olkoot $\eta > 0$ ja $x_0 = 0$. Määritellään rekursiivisesti $x_{k+1} = x_k - \eta f'(x_k)$, kun $k \ge 0$.

(a) Osoita, että $f(x_{k+1}) < f(x_k)$, jos $f'(x_k) \neq 0$ ja $\eta < 1/C$.

(b) Toteuttaako funktio $f(x) = x^2 + 1$ epäyhtälön (1) jollakin $C > 0$? Perustele vastauksesi.

Ratkaisu.

(a) Sijoittamalla $x = x_{k+1}$ ja $y = x_k$ yhtälöön (1) saadaan

$$f(x_{k+1}) - f(x_k) - f'(x_k)(x_{k+1} - x_k) \leq C(x_{k+1} - x_k)^2.$$

Tämä on yhtäpitävää sen kanssa, että

$$f(x_{k+1}) \leq f(x_k) + f'(x_k)(x_{k+1} - x_k) + C(x_{k+1} - x_k)^2.$$

Rekursion $x_{k+1} = x_k - \eta f'(x_k)$ nojalla pätee

$$x_{k+1} - x_k = -\eta f'(x_k).$$

Sijoittamalla tämä edelliseen epäyhtälöön saadaan

$$\begin{aligned} f(x_{k+1}) &\leq f(x_k) + f'(x_k)\left[-\eta f'(x_k)\right] + C\left[-\eta f'(x_k)\right]^2 \\ &= f(x_k) - \eta f'(x_k)^2 + C\eta^2 f'(x_k)^2 \\ &= f(x_k) + (C\eta - 1)\eta f'(x_k)^2. \end{aligned}$$

Oletus $\eta < 1/C$ tarkoittaa, että $C\eta - 1 < C(1/C) - 1 = 1 - 1 = 0$. Toisin sanoen $\alpha := 1 - C\eta > 0$. Näin ollen

$$f(x_{k+1}) \leq f(x_k) + (C\eta - 1)\eta f'(x_k)^2 = f(x_k) - \alpha\eta f'(x_k)^2.$$

Koska tässä $\alpha > 0$, $\eta > 0$ ja $f'(x_k)^2 > 0$ oletuksen nojalla, saadaan

$$f(x_{k+1}) \leq f(x_k) - \alpha\eta f'(x_k)^2 < f(x_k),$$

mikä on haluttu epäyhtälö. Tämä osoittaa, että gradienttimenetelmän jokainen askel pienentää funktion f arvoa, jos funktio f toteuttaa yhtälön (1), mikä ei useinkaan toteudu.

(b) Funktiolle $f(x) = x^2 + 1$ pätee $f'(x) = 2x$. Näin ollen

$$\begin{aligned} f(x) - f(y) - f'(y)(x - y) &= (x^2 + 1) - (y^2 + 1) - 2y(x - y) \\ &= x^2 - y^2 - 2xy + 2y^2 \\ &= x^2 - 2xy + y^2 \\ &= (x - y)^2 \end{aligned}$$

Siten yhtälö (1) toteutuu kaikilla $C \geq 1$.

□

Harjoitus 7.4. Käytä automaattista derivointia valitsemallasi ohjelmointikielellä ja piirrä seuraavan funktion derivaatta välillä $[0, 1]$:

$$f(x) = \sin(\cos(\sin(\cdots\cos(x)\cdots))) = (\sin \circ \cos \circ \sin \circ \cdots \circ \sin \circ \cos)(x),$$

missä 50 siniä ja 50 kosinia vuorottelevat. Voit joko koodata automaattisen derivoinnin itse tai käyttää valmista kirjastoa.

Ratkaisu. Aluksi toteutetaan automaattinen derivointi itse lähtien luentomuistiinpanoissa annetusta esimerkkikoodista. Tässä se esitetään pelkistetyssä muodossa, josta on säilytetty vain harjoituksen ratkaisemiseen tarvittavat osat. Tarkempi selitys taustalla olevasta periaatteesta löytyy luentomuistiinpanoista.

```
import numpy as np
import matplotlib.pyplot as plt


# automatic differentiation forward-mode
class fDf:
    def __init__(self, val, deriv):
        self.val = val
        self.deriv = deriv


def sin(u: fDf) -> fDf:
    return fDf(np.sin(u.val), np.cos(u.val) * u.deriv)


def cos(u: fDf) -> fDf:
    return fDf(np.cos(u.val), -np.sin(u.val) * u.deriv)


# f(x) = sin(cos(sin(...cos(x)...))) nested 50 times with a loop
def f(x):
    for _ in range(50):
        x = sin(cos(x))
    return x


# x in [0, 1]
x = np.r_[0:1:101j] # u(x) = x
dx = 1 # u'(x) = 1


# derivative of f at x
dfx = f(fDf(x, dx)).deriv # Df(x)


# plot
plt.plot(x, dfx)
plt.title(r"$Df(x)$")
plt.xlabel(r"$x$")
plt.grid()
plt.show()
```

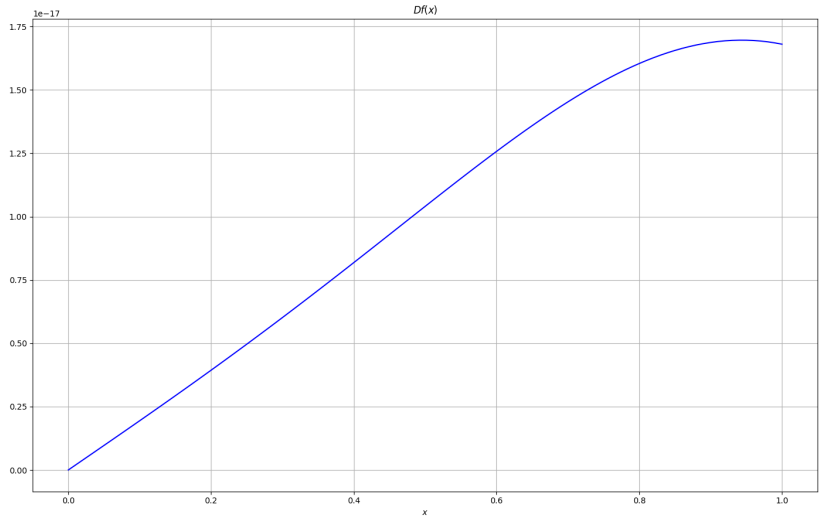
Käytännössä yleensä käytettäisiin `JAX`-Python-kirjastoa, joka tarjoaa tehokkaan (ja kätevän) toteutuksen automaattisesta derivoinnista *käänteismoodissa* `grad`-funktion kautta, jonka käytöstä löytyy perusteellinen selitys dokumentaation osiosta The Autodiff Cookbook. Tässä on lyhyt johdanto mukavuuden vuoksi:

> `grad` ottaa funktion ja palauttaa funktion. Jos sinulla on Python-funktio `f`, joka evaluoi matemaattisen funktion f, niin `grad(f)` on Python-funktio, joka evaluoi matemaattisen funktion ∇f. Tämä tarkoittaa, että `grad(f)(x)` edustaa arvoa $\nabla f(x)$.

Huomaa, että `jax.grad` olettaa, että `f` on skalaariarvoinen funktio, joten `grad(f)` pystyy evaluoimaan vain yhden pisteen x kerrallaan. Laskeaksemme $\nabla f(x)$:n useille syötteille kerralla (eli `Array`-oliolle) voimme käyttää `vmap`-funktiota, joka palauttaa Python-funktion vektoroidun kuvauksen; voit lukea lisää automaattisesta vektoroinnista `JAX`:n dokumentaatiosta.

```
import jax.numpy as jnp
from jax import grad, vmap
import matplotlib.pyplot as plt

def f(x):
    for _ in range(50):
        x = jnp.sin(jnp.cos(x))
    return x

df = vmap(grad(f)) # derivative of f (vectorised)

x = jnp.r_[0:1:101j]
dfx = df(x) # Df(x)

plt.plot(x, dfx)
plt.title(r"$Df(x)$")
plt.xlabel(r"$x$")
plt.grid()
plt.show()
```

`grad` ja `vmap` ovat esimerkkejä siitä, mitä `JAX` kutsuu "yhdistettäviksi muunnoksiksi" (composable transformations), eli operaattoreista, joita voidaan yhdistellä ja soveltaa mihin tahansa funktionaalisesti puhtaaseen Python-funktioon.



$Df(x)$ (eli `dfx` ohjelmamme yhteydessä) ja sen kuvaaja ovat samat riippumatta siitä, toteutatko automaattisen derivoinnin itse vai käytät valmista kirjastoa (tietysti samoille syötepisteille `x`), koska se on määritelmältään eksakti.

□

Harjoitus 7.5. Toteuta valitsemallasi ohjelmointikielellä luentomuistiinpanojen osiossa 7 kuvattu yhden muuttujan gradienttimenetelmä.

Piirrä seuraavat funktiot välillä $[-5,5]$ ja merkitse niiden lokaalit ja globaalit minimit:

$$f(x) = x^2 - 8(\cos(x))^2 \quad \text{ja} \quad f(x) = x^2 + 2(\sin(x))^2.$$

Sovella toteuttamaasi algoritmia näihin kahteen funktioon parametreilla $\eta = 0.05$, $t = 10^{-4}$, $n_{\max} = 10^3$ ja $x_0 \in \{1,2,3,4,5\}$. Kirjaa jokaiselle funktiolle f ja aloituspisteelle x_0

(i) algoritmisi palauttama minimipiste,
(ii) onko tämä piste lokaali vai globaali minimi ja
(iii) kuinka monta kertaa algoritmi laski derivaatan arvon.

Voit laskea derivaatat käsin tai käyttää automaattista derivointia.

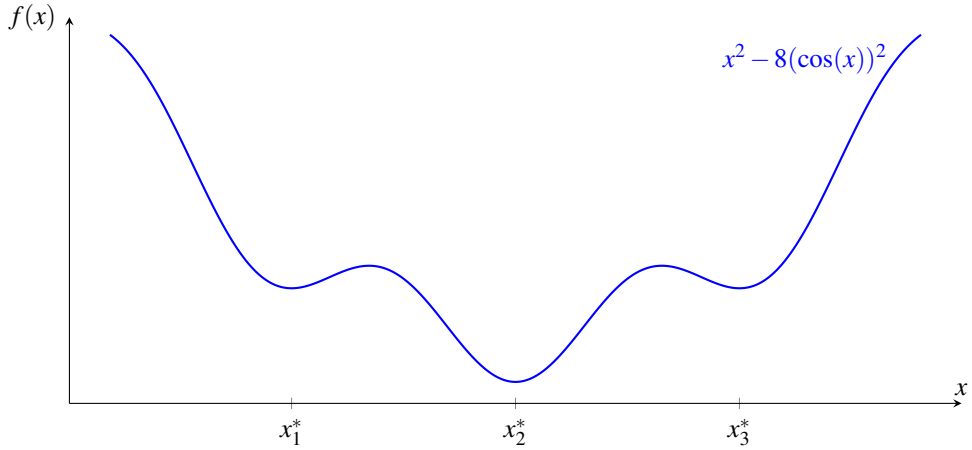
Ratkaisu. Käytämme jälleen erinomaista `JAX`-Python-kirjastoa automaattisen derivoinnin suorittamiseen käänteismoodissa. Seuraava koodi toteuttaa luentomuistiinpanoissa esitellyn yhden muuttujan gradienttimenetelmän, jossa iteraatiokaava on

$$x_{k+1} = x_k - \eta f'(x_k) \tag{2}$$

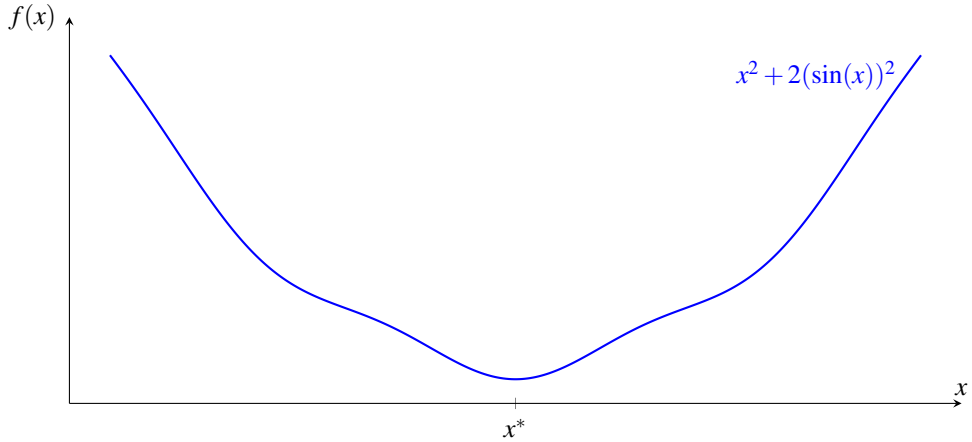
ja silmukka päättyy $n_{\max} = 10^3$ iteraation jälkeen tai kun $\left|f'(x_k)\right| < t = 10^{-4}$.

```python
import jax.numpy as jnp
from jax import grad


# univariate gradient descent
def unigd(f, x: float, learnrate=0.05, tol=1e-4, maxsteps=10**3):
    df = grad(f) # autodiff

    # iterate
    for n in range(maxsteps): # exit condition based on max iterations
        if abs(dfx := df(x)) < tol: # exit condition based on precision
            # terminate when f'(x) is small enough
            break
        else:
            # gradient descent step
            x -= learnrate * dfx

    # results
    return x, n


# definitions
x0 = jnp.r_[1.0:6]
f1 = lambda x: x**2 - 8 * jnp.cos(x) ** 2
f2 = lambda x: x**2 + 2 * jnp.sin(x) ** 2


# solve
for f in (f1, f2):
    print()
    for x in x0:
        x_min, n = unigd(f, x)
        print(f'{x=:.16} -> x_{n}={x:.6f}')
```

Tarkastellaan funktioitamme lähemmin. Meidän on paikallistettava ja luokiteltava niiden minimit, minkä voimme tehdä analysoimalla niiden kuvaajia.



Funktiolla $f(x) = x^2 - 8(\cos(x))^2$ on kolme lokaalia minimipistettä, $x_1^* \approx -2.76072$, $x_2^* = 0$ ja $x_3^* \approx 2.76072$. Näistä $x_0^* = 0$ on yksikäsitteinen globaali minimipiste.



Funktiolla $f(x) = x^2 + 2(\sin(x))^2$ on vain yksi lokaali minimipiste, $x^* = 0$. Se on myös yksikäsitteinen globaali minimipiste.

Alla olevasta taulukosta (**vasemmalla**) näkyvät mallikoodilla saadut tulokset funktiolle $f(x) = x^2 - 8(\cos(x))^2$ kullekin aloituspisteelle x_0. Tässä $x_{\min}$ on gradienttimenetelmän palauttama minimipiste, n on derivaatan arvon laskentojen lukumäärä ja "tyyppi" on minimipisteen $x_{\min}$ tyyppi (lokaali vai globaali). Arvoilla $x_0 \in \{2,3,4,5\}$ algoritmi löytää vain lokaalin minimipisteen. Kuitenkin arvolla $x_0 = 1$ se pystyy löytämään globaalin minimipisteen.

Vastaava taulukko (**oikealla**) on funktiolle $f(x) = x^2 + 2(\sin(x))^2$. Algoritmi löytää aina globaalin minimipisteen, mikä ei ole yllättävää, koska se on funktion ainoa kriittinen piste.

$f(x) = x^2 - 8(\cos(x))^2$

x_0	*(i)* $x_{\min}$	*(ii)* tyyppi	*(iii)* n
1.0	0.000001	globaali	7
2.0	2.760719	lokaali	14
3.0	2.760728	lokaali	9
4.0	2.760730	lokaali	10
5.0	2.760727	lokaali	13

$f(x) = x^2 + 2(\sin(x))^2$

x_0	*(i)* $x_{\min}$	*(ii)* tyyppi	*(iii)* n
1.0	0.000016	globaali	32
2.0	0.000014	globaali	39
3.0	0.000013	globaali	45
4.0	0.000015	globaali	47
5.0	0.000015	globaali	49

Alla esittelemme *vektoroidun* version yhden muuttujan gradienttimenetelmästä, joka pystyy evaluoimaan syötejoukon `x0` yhdellä kertaa (*array-ohjelmointi*-tyyliin).

Emme voi käyttää Pythonin tavallisia ohjausrakenteita ja loogisia operaattoreita (kuten `if`, `else`, `break`, `and`, `or`, ...) sellaisen funktion sisällä, jonka haluamme vektoroida. Tämä johtuu siitä, että `JAX` vaatii, että funktio on *funktionaalisesti puhdas* (ks. tehtävän 7.4 malliratkaisu). Siksi käytämme `jax.lax.while_loop`-funktiota (ks. dokumentaatiosta `while_loop`:n semantiikka Pythonin `itertools`-tyyliin) iteraatiomme uudelleenkirjoittamiseen; `_unigd` on muutoin vastaava kuin tehtävän 7.5 `unigd`. Nyt `JAX`:n näkökulmasta funktio `_unigd` on funktionaalisesti puhdas ja voidaan onnistuneesti "yhdistää", eli vektoroida tai JIT-kääntää.

Käytämme `jax.vmap`:iä `_unigd`:n vektorointiin "syöteakselin" `x0` suhteen: `in_axes`-argumentti määrittää paikka paikalta, mitkä `_unigd`:n argumenteista tulee vektoroida, eli `None` indekseillä 0, 2, 3, 4 kertoo `jax.vmap`:lle, että `_unigd`:n argumentit paikoissa 1, 3, 4, 5 (eli `f`, `learnrate`, `tol` ja `maxsteps`) ovat skalaareja, kun taas 0 indeksillä 1 kertoo, että toinen syöteargumentti (eli `x0`) on `Array` ja tulee vektoroida 0-akselin suhteen (eli Pythonin indeksoinnissa array:n ensimmäisen dimension suhteen). Olemme toteuttamassa *yhden muuttujan* gradienttimenetelmää, joten on kohtuullista olettaa, että `x0` on yksiulotteinen array, jonka jokainen alkio edustaa algoritmimme (2) yhteydessä eri x_0:aa.

`vectorised_unigd` ei funktionaalisesti puhtaana säilyttänyt oletusarvoja, jotka on määritelty `_unigd`:n variadisessa määrittelyssä (jotka säilytimme siellä mukavuuden vuoksi siltä varalta, että haluaisimme käyttää `_unigd`:tä debuggaustarkoituksiin). Siksi käärimme `vectorised_unigd`:n toiseen funktioon `unigd`, joka toteuttaa oletusarvomme uudelleen. Lopulta meillä on kätevä funktio `unigd`, joka on vektoroitu ja voi siten operoida usealla x_0:lla samanaikaisesti.

```
import jax
import jax.numpy as jnp

jnp.set_printoptions(suppress=True, precision=6)


# constants
ETA = 0.05
T = 1e-4
N_MAX = 1000


# univariate gradient descent
def _unigd(f, x0, learnrate=ETA, tol=T, maxsteps=N_MAX):
    df = jax.grad(f)

    def cond_fun(state):
        x, n = state
        return jnp.logical_and((jnp.abs(df(x)) >= tol), (n < maxsteps))

    def body_fun(state):
        x, n = state
        return x - learnrate * df(x), n + 1

    init_val = (x0, n := 0)
    x_min, *_ = jax.lax.while_loop(cond_fun, body_fun, init_val)

    return x_min


# vectorise along the second input argument
vectorised_unigd = jax.vmap(_unigd, in_axes=(None, 0, None, None, None))
```

```python
# wrapper with variadic definiton (i.e. with defaults)
def unigd(f, x0, learnrate=ETA, tol=T, maxsteps=N_MAX):
    return vectorised_unigd(f, x0, learnrate, tol, maxsteps)


# definitions
x0 = jnp.r_[1.0:6]
f1 = lambda x: x**2 - 8 * jnp.cos(x) ** 2
f2 = lambda x: x**2 + 2 * jnp.sin(x) ** 2


# solve
for f in (f1, f2):
    x_min = unigd(f, x0)
    print(jnp.c_[x0, x_min])
```

Vektorointi tuottaa merkittäviä tehokkuushyötyjä: yllä oleva mallikoodi suoriutuu 5–10 kertaa nopeammin kuin aiempi versio.

□